

API 仕様書

サービス名: 決済メール解析による自動家計簿サービス 言語 / フレームワーク: Go / Gin + GORM 対応する仕様書: [step2-db-spec.md](#) / [step2-er.md](#)

ドラフト版。全体仕様とエンドポイント一覧を確定させ、詳細仕様は**代表的な 6 エンドポイント**を先行して記載している。残りはメンタリングで粒度と形式を確認したうえで同じ形式で展開する。

1. API の概要

決済通知メールから自動抽出した支出データを提供する REST API。

扱う主なデータは以下の 4 つ。

データ	説明
取引 (transaction)	名寄せ後の実際の支出 1 件
決済イベント (payment_event)	メールから抽出した生の決済 1 件。取引の構成要素
店舗 (merchant)	正規化済みの店舗。住所・座標を持つ
通知 (notification)	即時通知・予算超過通知・月次サマリの送信履歴

メールの取得・解析・名寄せ・分類はバックグラウンドのバッチが行い、本 API は **その結果の参照と訂正**、および設定の管理を担当する。

2. 共通仕様

ベース URL

```
http://<your-host>/api/v1
```

日時フォーマット

ISO 8601 (例: **2026-09-09T12:34:56+09:00**)。タイムゾーンオフセットを必ず含む。

金額

すべて **最小単位の整数** (円の場合 1 = 1 円) で表現する。小数は用いない。

```
{ "amount_minor": 580, "currency": "JPY" }
```

ページネーション

一覧系のエンドポイントは以下のクエリパラメータを受け付ける。

パラメータ	型	既定値	説明
limit	integer	20	取得件数（最大 100）
offset	integer	0	取得開始位置

レスポンスの **meta** に総件数を含める。

レスポンス形式

成功時:

```
{ "data": { }, "meta": { } }
```

エラー時:

```
{ "error": { "code": "ERROR_CODE", "message": "エラーメッセージ" } }
```

3. 認証方式

本サービスの認証は **2 層に分かれる**。混同しないよう明確に分離する。

層	対象	方式
アプリ 認証	この API を呼 ぶ権利	Google OAuth でログイン後に発行する Bearer トークン（JWT / HS256、有効期限 24 時間）
メール連 携認証	ユーザのメー ルを読む権利	Google OAuth（readonly スcope）等。取得した認証情報は暗号化して mail_accounts に保存し、API のリクエストには使わない

ヘルスチェックと OAuth の開始・コールバックを除く、すべてのリクエストに Bearer トークンが必要。

```
Authorization: Bearer <access_token>
```

4. ステータスコード一覧

ステータスコード	意味
200	リクエスト成功
201	リソース作成成功
202	非同期処理を受け付けた（同期の手動実行など）

ステータスコード	意味
204	成功、返却するコンテンツなし
400	バリデーションエラー / ビジネスロジックエラー
401	認証エラー（トークン不正・期限切れ）
403	権限エラー（他ユーザのリソースへのアクセス）
404	リソースが見つからない
409	競合（重複登録、状態不整合）
500	サーバ内部エラー

5. エンドポイント一覧

No.	API ID	API 名	メソッド	エンドポイント	認証
1	API-001	ヘルスチェック	GET	/health	不要
2	API-002	Google OAuth 開始	GET	/api/v1/auth/google	不要
3	API-003	Google OAuth コールバック	GET	/api/v1/auth/google/callback	不要
4	API-004	現在のユーザ取得	GET	/api/v1/auth/me	必要
5	API-005	ログアウト	POST	/api/v1/auth/logout	必要
6	API-006	連携メールアカウント一覧	GET	/api/v1/mail-accounts	必要
7	API-007	連携解除	DELETE	/api/v1/mail-accounts/{id}	必要
8	API-008	手動同期の実行	POST	/api/v1/mail-accounts/{id}/sync	必要
9	API-009	同期履歴取得	GET	/api/v1/sync-jobs	必要
10	API-010	取引一覧取得	GET	/api/v1/transactions	必要
11	API-011	取引の手動登録	POST	/api/v1/transactions	必要

No.	API ID	API 名	メソッド	エンドポイント	認証
12	API-012	取引詳細取得	GET	/api/v1/transactions/{id}	必要
13	API-013	取引の修正	PUT	/api/v1/transactions/{id}	必要
14	API-014	取引の削除	DELETE	/api/v1/transactions/{id}	必要
15	API-015	取引のマージ（名寄せ）	POST	/api/v1/transactions/{id}/merge	必要
16	API-016	マージ解除	POST	/api/v1/transactions/{id}/unmerge	必要
17	API-017	未解析メール一覧	GET	/api/v1/inbox/unparsed	必要
18	API-018	決済メール判定の教示	POST	/api/v1/inbox/{id}/classify	必要
19	API-019	カテゴリー一覧	GET	/api/v1/categories	必要
20	API-020	カテゴリー作成	POST	/api/v1/categories	必要
21	API-021	カテゴリー更新 / 削除	PUT / DELETE	/api/v1/categories/{id}	必要
22	API-022	店舗一覧 / 修正	GET / PUT	/api/v1/merchants[/]{id}]	必要
23	API-023	月次サマリ取得	GET	/api/v1/summaries/monthly	必要
24	API-024	サブスク一覧	GET	/api/v1/subscriptions	必要
25	API-025	サブスク状態更新	PUT	/api/v1/subscriptions/{id}	必要
26	API-026	予算の取得 / 設定	GET / PUT	/api/v1/budgets	必要
27	API-027	通知設定の取得 / 更新	GET / PUT	/api/v1/notification-settings	必要
28	API-028	通知履歴一覧	GET	/api/v1/notifications	必要

削除系エンドポイントの挙動

DELETE は「クライアントから見てそのリソースを消す」という意味であり、**サーバ側の物理削除とは一致しない**。本サービスでは以下のとおり扱う。

エンドポイント	サーバ側の処理	ステータス
API-007 連携解除	<code>mail_accounts.status = 'disabled'</code> に更新し <code>credential_encrypted</code> を破棄。 行は削除しない （メールの生データを CASCADE で失わないため）	204
API-014 取引の削除	<code>transactions.deleted_at = now()</code> の 論理削除 。物理削除すると決済イベントが未名寄せに戻り、次のバッチで同じ取引が再生成されるため	204
API-021 カテゴリ削除	物理削除。参照している取引の <code>category_id</code> は FK により NULL になる	204

詳細は [step2-db-spec.md](#) の 3.2 / 3.7 の削除方針を参照。

詳細仕様を先行して記載する**エンドポイント**（API の形をひとつとおり網羅するもの）

API ID	選定理由
API-004	認証が必要な最小のエンドポイント
API-010	一覧系（ページネーション + 複数条件の絞り込み）
API-011	登録系（バリデーション + 201）
API-013	更新系（副作用としてカテゴリ規則を学習する）
API-015	CRUD に収まらないアクション型エンドポイント
API-023	集計系（DB のレコードをそのまま返さない）

6. 各 API の詳細仕様

API-004 現在のユーザ取得

```
GET /api/v1/auth/me
```

リクエストパラメータ: なし

レスポンス (200 OK)

フィールド名	型	説明
data.id	integer	ユーザ ID

フィールド名	型	説明
data.email	string	メールアドレス
data.display_name	string / null	表示名
data.timezone	string	集計の基準タイムゾーン
data.mail_accounts_count	integer	連携済みメールアカウント数
data.needs_reauth	boolean	再認証が必要な連携があるか

レスポンス例

```
{
  "data": {
    "id": 1,
    "email": "user@example.com",
    "display_name": "楊金澤",
    "timezone": "Asia/Tokyo",
    "mail_accounts_count": 1,
    "needs_reauth": false
  }
}
```

エラーレスポンス

ステータスコード	エラーコード	エラー内容
401	INVALID_TOKEN	トークンが不正
401	TOKEN_EXPIRED	トークンの有効期限切れ

API-010 取引一覧取得

```
GET /api/v1/transactions
```

リクエストパラメータ (クエリ)

パラメータ名	型	必須	説明
from	string	-	対象期間の開始日 (YYYY-MM-DD、JST 基準)
to	string	-	対象期間の終了日 (YYYY-MM-DD、JST 基準)
category_id	integer	-	カテゴリで絞り込む
payment_method_id	integer	-	決済手段で絞り込む
merchant_id	integer	-	店舗で絞り込む

パラメータ名	型	必須	説明
needs_review	boolean	-	<code>true</code> の場合、疑似重複フラグが立っている取引のみ
q	string	-	店舗名・メモの部分一致検索
limit	integer	-	取得件数（既定 20、最大 100）
offset	integer	-	取得開始位置（既定 0）

レスポンス (200 OK)

フィールド名	型	説明
data[].id	integer	取引 ID
data[].occurred_at	string	決済日時 (ISO 8601)
data[].amount_minor	integer	金額 (最小単位)
data[].currency	string	通貨コード
data[].merchant	object / null	店舗情報
data[].merchant.id	integer	店舗 ID
data[].merchant.name	string	店舗名
data[].merchant.address	string / null	住所 (ジオコーディング済みの場合)
data[].category	object / null	カテゴリ情報
data[].payment_method	object / null	決済手段
data[].source	string	<code>email</code> / <code>manual</code>
data[].status	string	<code>pending</code> / <code>confirmed</code>
data[].is_possible_duplicate	boolean	疑似重複の確認待ちか
data[].event_count	integer	紐づく決済イベント数
meta.total	integer	条件に合致する総件数
meta.total_amount_minor	integer	条件に合致する取引の合計金額

レスポンス例

```
{
  "data": [
    {
      "id": 1024,
      "occurred_at": "2026-09-09T12:31:00+09:00",
      "amount_minor": 580,
      "currency": "JPY",
      "merchant": {
        "id": 42,
```

```
    "name": "セブン-イレブン渋谷道玄坂店",
    "address": "東京都渋谷区道玄坂1-1-1"
  },
  "category": { "id": 3, "name": "食費" },
  "payment_method": { "id": 2, "name": "楽天カード", "kind":
"credit_card" },
  "source": "email",
  "status": "confirmed",
  "is_possible_duplicate": false,
  "event_count": 1
}
],
"meta": { "total": 187, "total_amount_minor": 143250 }
}
```

エラーレスポンス

ステータスコード	エラーコード	エラー内容
400	INVALID_DATE_RANGE	from が to より後
400	LIMIT_TOO_LARGE	limit が 100 を超えている
401	INVALID_TOKEN	トークンが不正

API-011 取引の手動登録

現金など、通知メールが発生しない決済を登録する。決済イベントを経由せず **transactions** に直接作成される (**source = manual**)。

POST /api/v1/transactions

リクエストパラメータ (ボディ)

パラメータ名	型	必須	説明
occurred_at	string	○	決済日時 (ISO 8601)
amount_minor	integer	○	金額 (最小単位、1 以上)
currency	string	-	通貨コード (既定 JPY)
merchant_name	string	-	店舗名 (既存になれば新規作成する)
category_id	integer	-	カテゴリ ID
payment_method_id	integer	-	決済手段 ID
note	string	-	メモ (500 文字以内)

リクエスト例

```
{
  "occurred_at": "2026-09-09T19:20:00+09:00",
  "amount_minor": 1200,
  "merchant_name": "近所の定食屋",
  "category_id": 3,
  "payment_method_id": 5,
  "note": "現金"
}
```

レスポンス (201 Created)

フィールド名	型	説明
data.id	integer	作成された取引 ID
data.occurred_at	string	決済日時
data.amount_minor	integer	金額
data.merchant	object / null	店舗（新規作成された場合を含む）
data.category	object / null	カテゴリ
data.source	string	常に <code>manual</code>
data.is_user_edited	boolean	常に <code>true</code> （再計算の対象外）

エラーレスポンス

ステータスコード	エラーコード	エラー内容
400	VALIDATION_ERROR	必須項目の欠落、形式不正
400	INVALID_AMOUNT	金額が 0 以下
404	CATEGORY_NOT_FOUND	指定したカテゴリが存在しない
404	PAYMENT_METHOD_NOT_FOUND	指定した決済手段が存在しない

API-013 取引の修正

副作用を持つ点が重要。 カテゴリを変更した場合、その店舗のブランドに対する `category_rules` を upsert し、以降の同ブランドの取引が自動で正しく分類されるようにする。また、修正した取引は `is_user_edited = true` となり、名寄せの再計算対象から外れる。

```
PUT /api/v1/transactions/{id}
```

リクエストパラメータ (パス)

パラメータ名	型	必須	説明
id	integer	○	取引 ID

リクエストパラメータ（ボディ）

パラメータ名	型	必須	説明
occurred_at	string	-	決済日時
amount_minor	integer	-	金額（最小単位）
merchant_id	integer	-	店舗 ID
category_id	integer	-	カテゴリ ID
payment_method_id	integer	-	決済手段 ID
note	string	-	メモ
learn_category	boolean	-	true （既定）でカテゴリ規則を学習する。 false でこの取引のみ変更

リクエスト例

```
{
  "category_id": 7,
  "learn_category": true
}
```

レスポンス（200 OK）

フィールド名	型	説明
data.id	integer	取引 ID
data.category	object	更新後のカテゴリ
data.category_source	string	常に user
data.is_user_edited	boolean	常に true
meta.learned_rule	object / null	学習したカテゴリ規則
meta.learned_rule.match_type	string	brand / merchant
meta.learned_rule.match_value	string	照合値
meta.affected_future	boolean	今後の取引に適用されるか

レスポンス例

```
{
  "data": {
    "id": 1024,
    "category": { "id": 7, "name": "日用品" },
    "category_source": "user",
    "is_user_edited": true
  },
  "meta": {
    "learned_rule": { "match_type": "brand", "match_value": "セブン-イレブン" },
    "affected_future": true
  }
}
```

エラーレスポンス

ステータスコード	エラーコード	エラー内容
400	VALIDATION_ERROR	形式不正
403	FORBIDDEN	他ユーザの取引
404	TRANSACTION_NOT_FOUND	取引が存在しない

API-015 取引のマージ（名寄せ）

疑似重複として提示された 2 件の取引を、同一の買い物として 1 件に統合する。統合先に決済イベントを付け替え、**統合元は論理削除する**（`deleted_at = now()`、`merged_into_id = 統合先の ID`）。どちらの金額を採用するかは `event_type` の確からしさに従う。

統合元を物理削除しないのは、API-016 `unmerge` で復元できるようにするため。 物理削除すると、統合元に入っていたメモやカテゴリの手修正が失われ、「マージを元に戻す」が成立しない。

```
POST /api/v1/transactions/{id}/merge
```

リクエストパラメータ（パス）

パラメータ名	型	必須	説明
id	integer	○	統合先の取引 ID

リクエストパラメータ（ボディ）

パラメータ名	型	必須	説明
source_transaction_id	integer	○	統合元の取引 ID（この取引は論理削除される）

リクエスト例

```
{ "source_transaction_id": 1025 }
```

レスポンス (200 OK)

フィールド名	型	説明
data.id	integer	統合後の取引 ID
data.amount_minor	integer	採用された金額
data.event_count	integer	統合後に紐づく決済イベント数
data.is_possible_duplicate	boolean	常に <code>false</code>
meta.merged_transaction_id	integer	論理削除された統合元の取引 ID (<code>unmerge</code> に渡す)
meta.amount_source_event_type	string	金額の採用元となったイベント種別

レスポンス例

```
{
  "data": {
    "id": 1024,
    "amount_minor": 3000,
    "event_count": 2,
    "is_possible_duplicate": false
  },
  "meta": {
    "merged_transaction_id": 1025,
    "amount_source_event_type": "settlement"
  }
}
```

エラーレスポンス

ステータスコード	エラーコード	エラー内容
400	SAME_TRANSACTION	統合元と統合先が同一
403	FORBIDDEN	他ユーザの取引
404	TRANSACTION_NOT_FOUND	いずれかの取引が存在しない
409	ALREADY_MERGED	統合元がすでに他の取引に統合されている (<code>merged_into_id</code> が設定済み)
409	ALREADY_DELETED	いずれかの取引がユーザによって削除済み (<code>deleted_at</code> が設定済み、かつマージ由来でない)

統合元を論理削除で残しているため、**409** の 2 つを状態から判定できる。物理削除にすると行が消えて **404** としか答えられず、「すでにマージ済み」なのか「そもそも存在しない」のかを区別できなくなる。

API-023 月次サマリ取得

DB のレコードをそのまま返さず、集計結果を返す。月末の配信メールと同じ内容を画面でも参照できるようにする。

```
GET /api/v1/summaries/monthly
```

リクエストパラメータ (クエリ)

パラメータ名	型	必須	説明
month	string	-	対象月 (YYYY-MM 、既定は当月。JST 基準)

レスポンス (200 OK)

フィールド名	型	説明
data.month	string	対象月
data.total_amount_minor	integer	当月の合計支出
data.prev_total_amount_minor	integer / null	前月の合計支出
data.diff_ratio	number / null	前月比 (1.0 で同額)
data.transaction_count	integer	取引件数
data.breakdown[].category	object	カテゴリ
data.breakdown[].amount_minor	integer	カテゴリ別の合計
data.breakdown[].ratio	number	全体に占める割合
data.active_subscriptions[]	array	継続中のサブスク
data.budget.amount_minor	integer / null	当月の予算
data.budget.used_ratio	number / null	予算の消化率
meta.is_complete	boolean	対象月が終了しているか (当月は false)

レスポンス例

```
{
  "data": {
    "month": "2026-08",
    "total_amount_minor": 143250,
    "prev_total_amount_minor": 128900,
```

```
{
  "diff_ratio": 1.11,
  "transaction_count": 187,
  "breakdown": [
    { "category": { "id": 3, "name": "食費" }, "amount_minor": 62400,
    "ratio": 0.44 },
    { "category": { "id": 7, "name": "日用品" }, "amount_minor": 31200,
    "ratio": 0.22 }
  ],
  "active_subscriptions": [
    { "id": 5, "merchant_name": "Netflix", "amount_minor": 1590, "cycle":
    "monthly" }
  ],
  "budget": { "amount_minor": 150000, "used_ratio": 0.96 }
},
"meta": { "is_complete": true }
}
```

エラーレスポンス

ステータスコード	エラーコード	エラー内容
400	INVALID_MONTH_FORMAT	month の形式が YYYY-MM でない
404	NO_DATA	対象月にデータが存在しない

7. 主な設計判断

- アクション型エンドポイントを用意した merge / unmerge / classify** は、リソースの状態を書き換えるだけでなく **副作用（イベントの付け替え、ルールの学習）を伴う**。汎用の **PUT** に含めると、どのフィールドを変えたかで挙動が変わり分かりにくくなるため、意図が明示されるアクション型のエンドポイントに分離した。
- PUT /transactions/{id} の副作用を meta で返す** カテゴリ修正時に学習したルールを **meta.learned_rule** として返すことで、「この修正が今後にも効く」ことを画面上でユーザに伝えられる。学習させたくない場合のために **learn_category** フラグも用意した。
- 金額を最小単位の整数で返す** クライアント側での丸め誤差を防ぐため。表示時の桁区切りはフロント側の責務とする。
- 一覧の meta に合計金額を含めた** 絞り込み条件を変えるたびに合計を別途取得する必要がなくなり、リクエスト数を削減できる。
- 認証を 2 層に分離した** API を呼ぶための認証と、メールを読むための認証は目的も有効期限も異なる。後者のトークンは API リクエストには一切使わず、暗号化して DB に保持する。
- バッチ処理の API を最小限にした** 取込・解析・名寄せはバッチが行うため、API 側は **POST /mail-accounts/{id}/sync** (202 で受付) と **GET /sync-jobs** (履歴参照) のみを提供する。
- DELETE の意味をエンドポイントごとに定義した DELETE** は「クライアントから見て消える」ことを表すだけで、サーバ側が物理削除するとは限らない。本サービスでは **transactions** は論理削除、**mail_accounts** は **status = 'disabled'**、**categories** は物理削除と使い分けており、その対

応表を 5 章に明記した。物理削除にすると、削除した取引が次の名寄せバッチで再生成される（詳細は DB 仕様書 3.7）。

8. **merge** の統合元を残すことで **unmerge** を成立させた 統合元を物理削除すると、そこに入っていたユーザの手修正が失われ、「マージを元に戻す」が実現できない。**merged_into_id** に統合先を記録して論理削除することで、両方の列を NULL に戻すだけで復元できるようにした。副次的に、**409 ALREADY_MERGED** を状態から判定できるようになっている。

8. 判断に迷った点（メンタリングで相談したい）

1. **merge** を統合先の下に置いたこと — **POST /transactions/{id}/merge** と **POST /transactions/merge**（両方をボディで指定）のどちらが適切か
2. 月次サマ리를 API で毎回集計するか、**monthly_summaries** に保存済みの結果を返すか
3. **GET /inbox/unparsed** の位置づけ — メールはリソースとして公開すべきか、「要対応キュー」という別の概念にすべきか
4. エラーコードの粒度 — **VALIDATION_ERROR** にまとめるか、項目ごとに分けるか
5. ページネーション方式 — offset 方式で始めたが、取引件数が増えた場合にカーソル方式へ移行すべきか